

CC2 – Rapport de mi-parcours : implémentation et résultats préliminaires

- Manolo SARDÓ
- Encadrant : Millian Poquet
- 20 avril 2026
- https://github.com/Manolo-dev/nat_set/

Introduction

Dans de nombreux domaines (systèmes d’exploitation, bases de données, moteurs de recherche, ici ce sera pour du scheduling), la gestion “*efficace*” (une première question se pose ici : qu’entend-on par “*efficace*” ? **temps d’exécution, mémoire utilisée, ou les deux** ?) d’ensembles d’entiers est cruciale pour les performances. Les opérations ensemblistes (union, intersection, différence) sont au coeur de ces applications, nécessitant des structures de données optimisées pour différents profils d’utilisation. Ici, nous cherchons à implémenter et évaluer plusieurs structures de données pour représenter des ensembles d’entiers de 32 bits, avec un focus sur les compromis entre temps d’exécution et mémoire utilisée.

Trait ResourceSet

Nous avons défini un trait Rust commun, `ResourceSet`, qui spécifie les opérations minimales attendues (certaines méthodes ont des implémentations par défaut pour faciliter le développement).

```
pub trait ResourceSet: Clone + PartialEq + std::fmt::Debug {
    fn new() -> Self;
    fn singleton(e: u32) -> Self;
    fn from_iter<I: IntoIterator<Item = u32>>(iter: I) -> Self {
        iter.into_iter().fold(Self::new(), |acc, e| acc.union(&Self::singleton(e)))
    }
    fn contains(&self, elem: u32) -> bool;
    fn len(&self) -> usize;
    fn is_empty(&self) -> bool { self.len() == 0 }
    fn union(&self, other: &Self) -> Self;
    fn intersection(&self, other: &Self) -> Self;
    fn difference(&self, other: &Self) -> Self;
    fn iter(&self) -> Box<dyn Iterator<Item = u32> + '_>;
    fn serialize(&self) -> String {
        String::from("") + &self.iter().map(|e| format!("{}", e)).collect::<Vec<_>>().join(", ") + ""
    }
    fn deserialize(s: &str) -> Self {
        Self::from_iter(s.split(',').filter_map(|part| part.trim().parse::<u32>().ok()))
    }
}
```

Travail effectué

Structures implémentées (ou en cours d’implémentation)

Quatre structures ont été complètement implémentées et testées :

- **NaiveListSet** : un vecteur trié sans doublons. Les opérations ensemblistes sont réalisées par fusion linéaire (coût $O(n + m)$). Simple mais inefficace pour les grands ensembles. Il servait surtout de point de départ pour vérifier le fonctionnement du trait et des tests.
- **HashSetSet** : wrapper autour de `std::collections::HashSet`. Très rapide pour les tests d’appartenance ($O(1)$ amorti), mais mémoire plus élevée et itération non ordonnée.
- **IntervalSet** : stocke les bornes des intervalles contigus dans un `BTreeSet`. La représentation est compacte pour les ensembles denses ou peu fragmentés. Les opérations ensemblistes sont réalisées par parcours simultané des intervalles (coût linéaire en nombre d’intervalles).

- **RoaringSet** : utilisation de la bibliothèque externe **roaring**. Cette structure hybride (bitsets compressés) offre un bon compromis temps/mémoire pour une large gamme de densités.

Une cinquième structure, **SzemerSet** (basée sur les ensembles de Szemerédi, i.e. unions de progressions arithmétiques sur des intervalles), a été explorée mais s’est avérée trop complexe à finaliser dans le temps imparti (nous expliquerons les raisons plus bas).

Validation fonctionnelle

Deux sortes de tests sont effectués : - des tests unitaires naïfs, simples et qui permettent de vérifier le fonctionnement des méthodes. - des tests basés sur les propriétés (en utilisant la bibliothèque **proptest**) pour vérifier les propriétés algébriques des opérations ensemblistes. Ces tests génèrent des ensembles et vérifient que les propriétés tiennent pour toutes les implémentations. Les propriétés vérifiées incluent : - Commutativité de union et intersection - Associativité - Distributivité de intersection sur union - Idempotence - Difference avec l’ensemble vide - Sérialisation/désérialisation (round-trip)

Benchmarks préliminaires

Des mesures de performance ont été effectuées avec **criterion.rs** sur deux tailles d’ensembles (10 et 1 000 éléments), pour les opérations **from_iter**, **contains**, **union**, **intersection** et **difference**. Les résultats sont présentés dans le tableau ci-dessous (temps en nanosecondes, valeurs indicatives) (**NaiveListSet** n’a pas été inclus dans les benchmarks, son but était de vérifier la validité du trait et des tests, et il est clairement inefficace pour les tailles testées) :

Opération	Taille	HashSetSet	IntervalSet	RoaringSet
contains	small	96.7 ns	87.8 ns	66.9 ns
contains	medium	10.3 µs	8.34 µs	14.0 µs
union	small	204 ns	256 ns	81.4 ns
union	medium	18.5 µs	12.2 µs	2.35 µs
intersection	small	177 ns	269 ns	71.4 ns
intersection	medium	21.2 µs	67.7 µs	2.11 µs
difference	small	216 ns	245 ns	73.2 ns
difference	medium	29.9 µs	64.0 µs	2.07 µs
from_iter	small	159 ns	1.97 µs	1.25 µs
from_iter	medium	13.9 µs	205 µs	1.19 ms

Note : les résultats sont indicatifs et peuvent varier en fonction de la machine, de la charge système, etc. Ils sont présentés ici à titre d’exemple pour illustrer les tendances observées.

Observations : - **RoaringSet** domine pour toutes les opérations sur **small**, et reste très compétitif sur **medium** (sauf **contains** où il est devancé par **IntervalSet**). - **IntervalSet** a un **contains** très rapide sur **medium** (meilleur que **HashSetSet**), mais ses opérations **intersection** et **difference** sont anormalement lentes (respectivement $67\mu s$ et $64\mu s$) – cela suggère une complexité plus élevée que prévu, probablement due à une implémentation non optimisée ou à une mauvaise utilisation des spécificités du langage. - **HashSetSet** a des performances honorables mais souffre sur **difference** ($30\mu s$) et sur **from_iter** ($13.9\mu s$), à cause des allocations répétées. - La construction (**from_iter**) de **RoaringSet** est la plus lente sur **medium** ($1.19ms$) car la bibliothèque **roaring** optimise pour les opérations ensemblistes, pas pour les insertions individuelles.

Limites méthodologiques : - Les benchmarks ont été exécutés sur une machine portable sans isolation CPU, d’où une certaine variabilité (les intervalles de confiance ne sont pas rapportés ici mais sont de l’ordre de $\pm 1-5\%$). - Le jeu de données utilisé (**small** = 0..10, **medium** = 0..1000) ne couvre que des ensembles contigus, ce qui favorise **IntervalSet** et pénalise artificiellement **HashSetSet** (peu de collisions). Une évaluation avec des ensembles fragmentés est nécessaire pour une comparaison réaliste.

Difficultés rencontrées

Méchanceté du compilateur Rust

Rust est un langage puissant et manifestement merveilleux, mais son apprentissage est une infâme corvée. Les erreurs de compilation sont souvent cryptiques (heureusement parfois le compilateur fournit des suggestions, mais elles sont parfois à côté de la plaque, ou parfois contradictoires), les notes et warnings sont suffisamment nombreux pour surcharger le buffer du terminal et les librairies changent du tout au tout d'une version à l'autre.

Complexité de SzemerSet

L'idée de représenter un ensemble comme une union d'intervalles à pas (début, pas, fin), est séduisante pour compresser des motifs réguliers. Cependant, plusieurs difficultés majeures sont apparues : - Non-unicité de la représentation : un même ensemble peut être décrit par plusieurs familles de progressions, rendant difficile l'égalité structurelle. Il faut donc choisir une forme canonique, mais chacune des pistes explorées posent des problèmes de complexité ou de modélisation. - Chevauchement des régions : les progressions issues de différents intervalles peuvent se chevaucher, obligeant à une fusion complexe (problème résolvable par certaines modélisations, mais au prix d'une complexité accrue). - Calcul des opérations ensemblistes : l'union, l'intersection et la différence exigent de manipuler des ensembles de progressions arithmétiques, avec des intersections de classes de congruence (nécessitant le théorème des restes chinois) et une inclusion-exclusion exponentielle en le nombre de progressions. - Coût en pratique : même pour un petit nombre de progressions, l'inclusion-exclusion ($O(2^k)$) est rédhibitoire.

Une implémentation partielle (contains, len) a été réalisée, mais les opérations union, intersection et difference n'ont pas pu être finalisées dans le temps. Cette piste est donc mise de côté pour la phase 1 ; elle pourra être réexaminée en groupe si un intérêt se manifeste. Le principe de cette implémentation est tout de même intéressant :

```
#[derive(Clone, PartialEq, Debug)]
pub struct SzemerSet {
    inner: BTreeMap<u32, Vec<(u32, u32)>>, // (start, steps = [(modd, reste), ...])
    // Quand Vec<(u32, u32)> est vide, c'est un trou
}
```

$$\bigcup_{s \in \{0\} \cup \text{keys}(\text{inner}) \setminus \{\text{last}\}} \{n \in \llbracket s, \text{succ}(s) - 1 \rrbracket \mid \exists (m, r) \in \text{inner}[s] : m \mid (n - r)\}$$

On retrouve bien la structure d'une union d'intervalles à pas, avec une organisation par points de départ (clé du BTreeMap) et des classes de congruence (modulo m , reste r) pour chaque intervalle. Cependant, la complexité de manipulation de ces structures est élevée, notamment pour les opérations ensemblistes.

Lenteur des benchmarks

Les mesures sur grande taille (100 000 éléments) étaient extrêmement lentes. Les causes identifiées sont : - Machine peu puissante / mal adaptée (processeur mobile, aucune isolation des cœurs ni fréquence fixe) (la machine possède un processeur Intel Core i7 9th gen, ce qui est plus que suffisant pour des benchmarks de ce type, mais l'absence d'isolation CPU et de fréquence fixe introduit une variabilité importante. De plus, la charge système peut modifier les résultats, d'où la nécessité d'exécuter les benchmarks dans un environnement contrôlé pour obtenir des mesures fiables). - Code de benchmark non optimisé : la création des ensembles à chaque itération alourdit inutilement les mesures.

Perspectives

Phase individuelle (techniquement terminée mais à finaliser)

- Refonte du protocole de benchmark :
 - Utiliser taskset et la fixation de la fréquence CPU pour améliorer la reproductibilité.
 - Générer des ensembles aléatoires avec différents niveaux de densité et de fragmentation (mesurée par le nombre de changements de valeur).
 - Inclure la mesure de la dé/sérialisation.

- **Extension éventuelle :** implémenter un arbre AVL ou rouge-noir (si le temps le permet), ou une structure issue de la veille bibliographique (skip list ordonnée, ou une variante de Roaring Bitmaps), éventuellement finir l'implémentation de SzemerSet en imaginant une modélisation plus efficace.

Phase collective

- Harmonisation du trait **ResourceSet** entre tous les membres.
- Étude comparative approfondie : visualisations (temps vs. taille, fragmentation, densité) et analyse statistique.
- Rédaction d'un article commun (template Typst/LaTeX) et préparation de la soutenance.

Échéancier mis à jour

Semaine	Objectifs (réalisés / à réaliser)
S1–S2	Formation Rust, définition du trait, HashSetSet, NaiveListSet
S3–S4	IntervalSet + corrections, début benchmarks
S5–S6	RoaringSet, proptests, exploration SzemerSet (non aboutie)
S7	Refonte des benchmarks, optimisation contains, rédaction CC3
S8	Rendu final (code + mini-article) – 20 mai (à confirmer)

Conclusion

Malgré des difficultés sur la structure avancée **SzemerSet** et des benchmarks préliminaires peu exploitables, la phase 1 a permis de livrer quatre implémentations fonctionnelles et rigoureusement testées. La suite consistera à améliorer la rigueur expérimentale et à étendre les comparaisons, en vue d'une restitution finale de qualité.